ASSIGNMENT 15.5

2303A51351
T.Harivarshith
Batch 20

Task 1 – Student Attendance API

Task:

Create a RESTful API for student attendance tracking.

Instructions:

- Endpoints required: o GET /attendance → View attendance records o POST /attendance → Mark attendance o PUT /attendance/{id} → Update attendance record o DELETE /attendance/{id} → Remove attendance entry • Ensure JSON-based request and response handling. Expected Output:
- API for maintaining and updating attendance records.

Prompt:

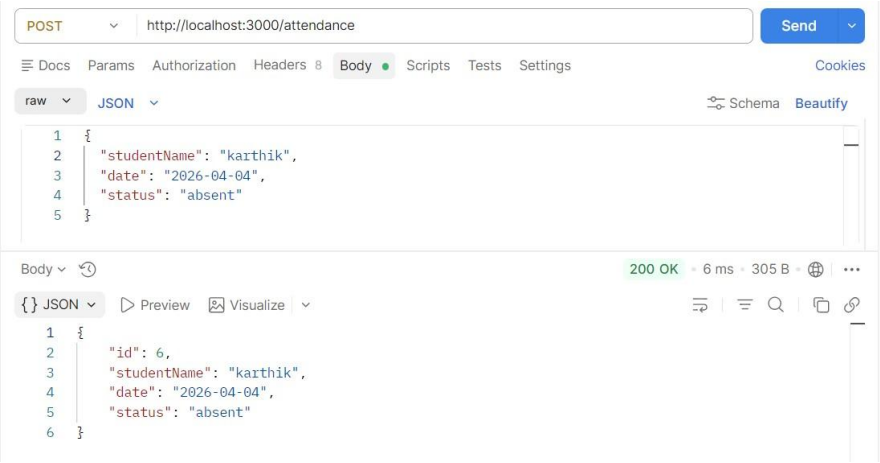
Create a RESTful API for student attendance tracking that supports operations to add, view, update, and delete attendance records. Ensure all requests and responses are handled using JSON format.

CODE:

```
const express = require("express");

const app = express();
const PORT = 3000;

app.use(express.json());

let attendance = [];

// GET - view all records
app.get("/attendance", (req, res) => {
res.json(attendance);
});

// POST - add attendance
app.post("/attendance", (req, res) => {
   const { studentName, date, status } = req.body;

   const newRecord = {
id: attendance.length + 1,
studentName, date,
status
   };

   attendance.push(newRecord);
   res.json(newRecord);
```

```
});

// PUT - update attendance
app.put("/attendance/:id", (req, res) => {
const id = parseInt(req.params.id);
  const record = attendance.find(r => r.id === id);

  if (!record) { return
res.status(404).send("Not found");
  }

  record.studentName = req.body.studentName || record.studentName;
  record.date = req.body.date || record.date;
  record.status = req.body.status || record.status;

  res.json(record);
});

// DELETE - remove attendance
app.delete("/attendance/:id", (req, res) => {
const id = parseInt(req.params.id);
  attendance = attendance.filter(r => r.id !== id);

  res.send("Deleted");
});

app.listen(PORT, () => { console.log(`Server running on
http://localhost:${PORT}`);
});
```

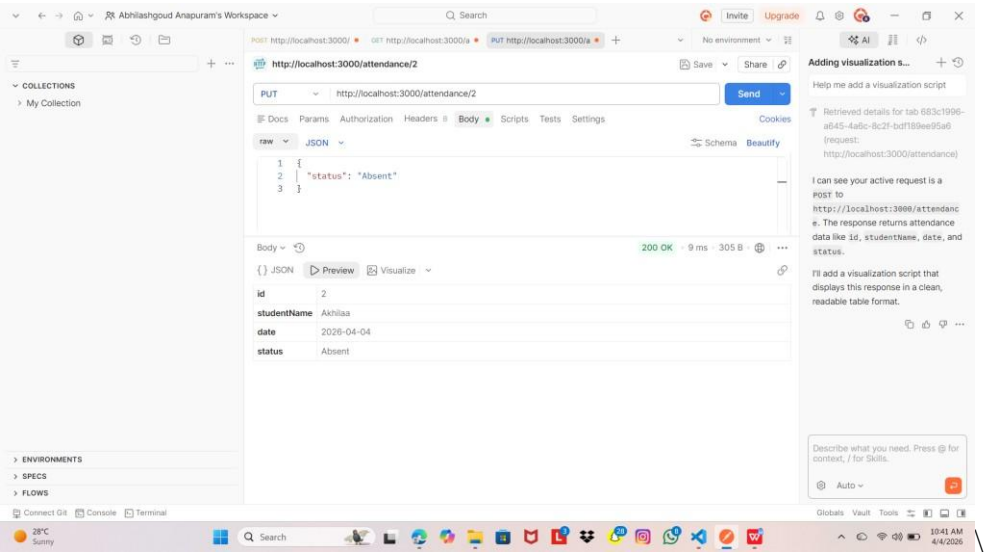
OUTPUT:

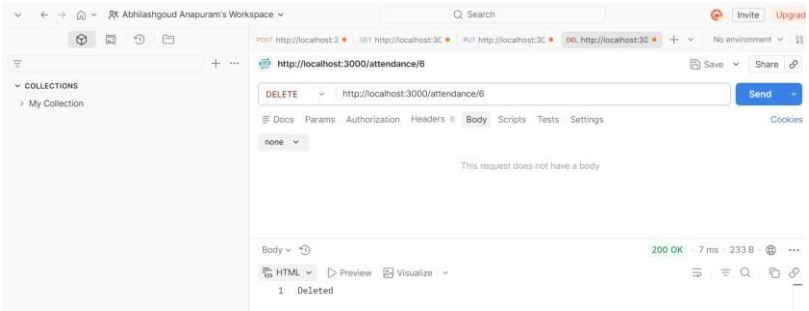


get

```
[
    {
        "id": 1,
        "studentName": "Ram",
        "date": "2026-04-04",
        "status": "Present"
    },
    {
        "id": 2,
        "studentName": "Akhilaa",
        "date": "2026-04-04",
        "status": "Present"
    },
    {
        "id": 3,
        "studentName": "poojitha",
        "date": "2026-04-04",
        "status": "Present"
    },
    {
        "id": 4,
        "studentName": "Arshad",
        "date": "2026-04-04",
        "status": "absent"
    },
    {
        "id": 5,
        "studentName": "karthik",
        "date": "2026-04-04",
        "status": "absent"
    },
    {
        "id": 6,
        "studentName": "karthik",
        "date": "2026-04-04",
        "status": "absent"
    }
]
```


	id	studentName	date	status
0	1	Ram	2026-04-04	Present
1	2	Akhilaa	2026-04-04	Present
2	3	poojitha	2026-04-04	Present
3	4	Arshad	2026-04-04	absent
4	5	karthik	2026-04-04	absent
5	6	karthik	2026-04-04	absent





Explanation:

A RESTful Student Attendance API allows users to manage attendance records by adding, viewing, updating, and deleting data using HTTP methods (GET, POST, PUT, DELETE).
It uses JSON format for sending and receiving data between the client and server.

Task 2 – Inventory Management API
Task:
Develop a REST API for warehouse inventory management. Instructions:
• Endpoints required:
o GET /inventory → List all items o POST /inventory → Add new item o PUT /inventory/{id} → Update stock quantity o DELETE /inventory/{id} → Remove an item Expected Output:
• API capable of managing inventory stock levels.

Prompt:

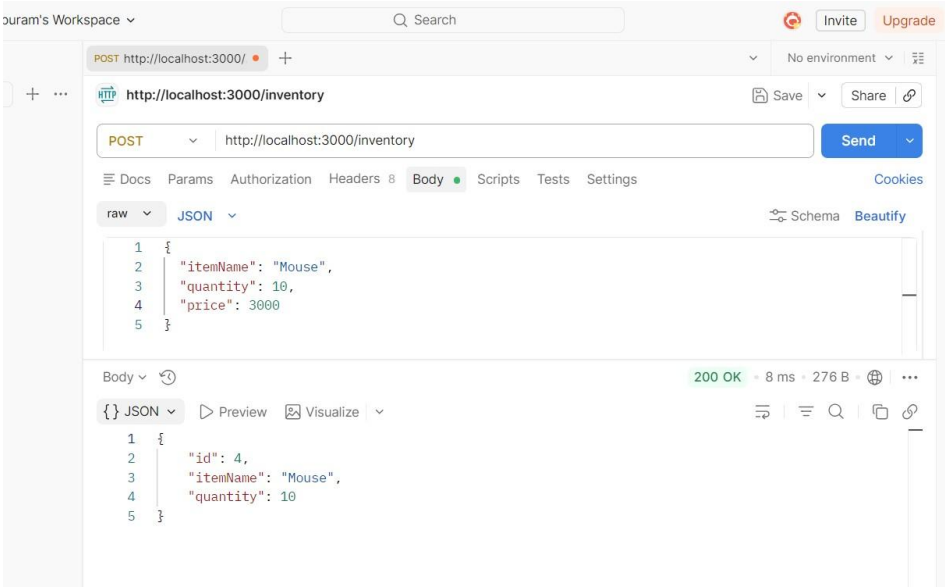
Create a RESTful API for warehouse inventory management that allows users to add, view, update, and delete inventory items. Ensure proper validation is implemented for stock values and all data is handled using JSON format.

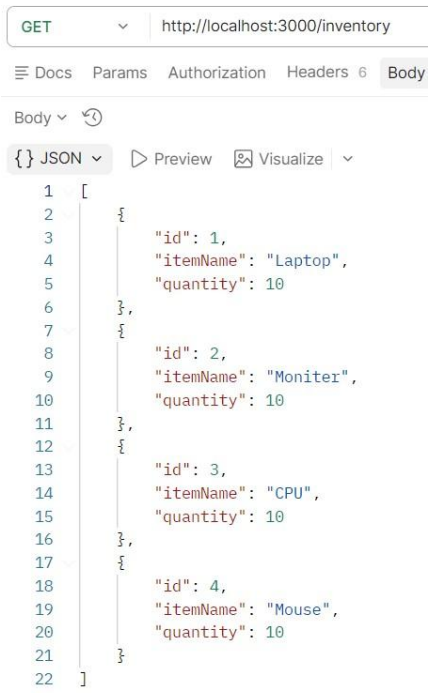
CODE:

```
app.put("/inventory/:id", (req, res) => {     const
id = parseInt(req.params.id);        const item =
inventory.find(i => i.id === id);

   if (!item) {
      return res.status(404).send("Item not found");
   }

   if (req.body.quantity != null && req.body.quantity < 0) {
return res.status(400).send("Invalid stock value");
   }
```

```
    if (req.body.price != null && req.body.price < 0) {
return res.status(400).send("Invalid price value");
    }

    item.quantity = req.body.quantity != null ? req.body.quantity : item.quantity;
item.price = req.body.price != null ? req.body.price : item.price;

    res.json(item);
});
```

OUTPUT:





	id	itemName	quantity
0	1	Laptop	10
1	2	Moniter	10
2	3	CPU	10
3	4	Mouse	10

PUT http://localhost:3000/inventory/2

Docs Params Authorization Headers 8 Body

raw JSON

```
1 {
2   "quantity": 5
3 }
```

Body

{} JSON Preview Visualize

```
1 {
2     "id": 2,
3     "itemName": "Moniter",
4     "quantity": 5
5 }
```

DELETE http://localhost:3000/inventory/3

Docs Params Authorization Headers 6 Body

Query Params

Key	Value
Key	Value

Body

HTML Preview Visualize

```
1 Item deleted
```

Explanation:

This Inventory Management API provides a centralized interface for tracking warehouse stock through standard CRUD operations (Create, Read, Update, Delete). By implementing built-in validation, the system ensures data integrity by preventing invalid entries, such as negative stock counts or empty item fields. **Task 3 – Hotel Room Booking API**

Task:
Develop a RESTful API for hotel room booking.
Instructions:
• Endpoints required:

o GET /rooms → View available rooms
o POST /rooms/book → Book a room o
GET /rooms/{id} → View booking
details o DELETE /rooms/{id} →
Cancel booking

- Implement error handling for unavailable rooms. Expected Output:
- API simulating hotel room booking operations.

PROMPT:

Develop a RESTful API for hotel room booking that allows users to view available rooms, book a room, check booking details by ID, and cancel a booking. The API should handle cases where rooms are unavailable and provide appropriate error messages. All requests and responses must use JSON format, enabling smooth communication between the client and server. This API will simulate basic hotel room booking operations and manage room availability efficiently.

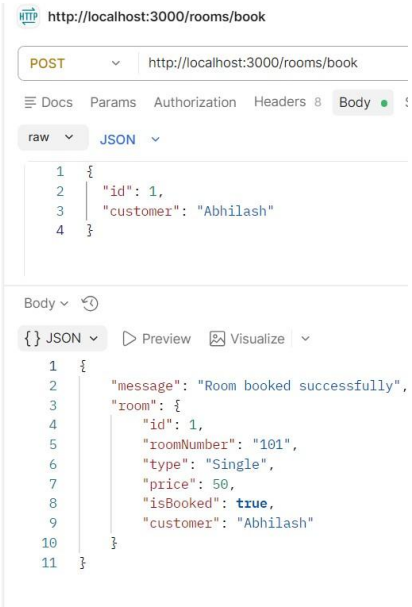
CODE:

```
const express = require("express");
const app = express();

app.use(express.json());

// Rooms data let
rooms = [
  { id: 1, roomNumber: "101", type: "Single", price: 50, isBooked: false, customer: null },
  { id: 2, roomNumber: "102", type: "Double", price: 80, isBooked: false, customer: null },
  { id: 3, roomNumber: "103", type: "Suite", price: 150, isBooked: false, customer: null }
];



// ✅ GET → View all rooms
app.get("/rooms", (req, res) => {
  res.json({
    message: "Rooms fetched successfully",
data: rooms
  });
});



// ✅ POST → Book a room
app.post("/rooms/book", (req, res) => {
const { id, customer } = req.body;

  if (!id || !customer) {
    return res.status(400).json({ error: "Room id and customer required" });
  }
```

```
  const room = rooms.find(r => r.id === id);

  if (!room) {     return res.status(404).json({ error: "Room
not found" });
  }

  if (room.isBooked) {
    return res.status(400).json({ error: "Room already booked" });
  }

  room.isBooked            =            true;
room.customer = customer;

  res.json({     message: "Room booked
successfully",     data: room
  });
});



// ✅ PUT → Update room details (no
restriction) app.put("/rooms/:id", (req, res) => {
const id = parseInt(req.params.id);     const {
customer, type, price } = req.body;

  const room = rooms.find(r => r.id === id);

  if (!room) {     return res.status(404).json({ error: "Room
not found" });
  }

  // update fields   if (customer !== undefined)
room.customer = customer;   if (type !== undefined)
room.type = type;   if (price !== undefined) room.price
= price;

  res.json({       message: "Room updated
successfully",
    data: room
  });
});


// ✅ DELETE → Cancel/reset
booking app.delete("/rooms/:id", (req,
```

```
res) => { const id = parseInt(req.params.id);

  const room = rooms.find(r => r.id === id);

  if (!room) {
    return res.status(404).json({ error: "Room not found" });
  }

  room.isBooked = false; room.customer = null;

  res.json({
    message: "Room reset successfully",
    data: room
  });
});


// ✅ START SERVER
app.listen(3000, () => {
  console.log("Server running on http://localhost:3000");
});
```

OUTPUT:



GET http://localhost:3000/rooms

Docs Params Authorization Headers 6 Body

Body

{} JSON Preview Visualize


```
[
    {
        "id": 1,
        "roomNumber": "101",
        "type": "Single",
        "price": 50,
        "isBooked": true,
        "customer": "Abhilash"
    },
    {
        "id": 2,
        "roomNumber": "102",
        "type": "Double",
        "price": 80,
        "isBooked": true,
        "customer": "Akhilaa"
    },
    {
        "id": 3,
        "roomNumber": "103",
        "type": "Suite",
        "price": 150,
        "isBooked": true,
        "customer": "poojitha"
    }
]
```


	id	roomNumber	type	price	isBooked	customer
0	1	101	Single	50	true	Abhilash
1	2	102	Double	80	true	Akhilaa
2	3	103	Suite	150	true	poojitha

PUT http://localhost:3000/rooms/1

Docs Params Authorization Headers 8 Body Sc

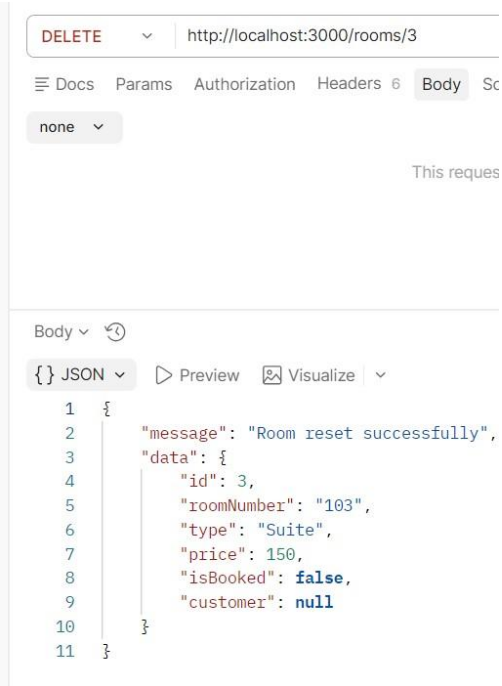
raw JSON


```
{
  "customer": "Rahul",
  "price": 120
}
```


Body

{} JSON Preview Visualize


```
{
    "message": "Room updated successfully",
    "data": {
        "id": 1,
        "roomNumber": "101",
        "type": "Single",
        "price": 120,
        "isBooked": false,
        "customer": "Rahul"
    }
}
```



Explanation:

The provided API uses an in-memory dictionary to track room availability and guest data across four distinct RESTful routes. It includes built-in logic to validate room IDs and prevent double-booking by checking the current status before confirming a reservation

Task 4 – Online Voting API

Task:
Create a RESTful API for an online voting system.
Instructions:

- Endpoints required:
  - GET /candidates → List candidates
  - POST /vote → Cast a vote
  - GET /results → View voting results
- Ensure one vote per user using in-memory validation. Expected Output:
- API simulating a secure online voting process.

PROMPT:

Create a RESTful API for an online voting system using Node.js and Express. The API should provide endpoints to list all candidates using GET /candidates, allow users to cast their vote using POST /vote, and display voting results using GET /results. The system must ensure that each user can vote only once by implementing inmemory validation, such as using a Set to track user IDs. The API should return proper JSON responses for both successful operations and error cases, simulating a simple and secure online voting process.

CODE:

```
const express = require("express");
const app = express();

app.use(express.json());
```

```
// Candidates let
candidates = [
  { id: 1, name: "Alice", votes: 0 },
  { id: 2, name: "Bob", votes: 0 },
  { id: 3, name: "Charlie", votes: 0 }
];

// Store voted users let
votedUsers = new Set();

// GET candidates
app.get("/candidates", (req, res) => {
res.json(candidates);
});

// POST vote
app.post("/vote", (req, res) => {
  const { userId, candidateId } = req.body;

  if (!userId || !candidateId) {
    return res.status(400).json({ error: "userId and candidateId required" });
}

  if (votedUsers.has(userId)) {
    return res.status(400).json({ error: "User already voted" });
  }

  const candidate = candidates.find(c => c.id === candidateId);
if (!candidate) {
    return res.status(404).json({ error: "Candidate not found" });
}

  candidate.votes += 1;
votedUsers.add(userId);

  res.json({
    message: "Vote cast successfully",
    candidate: { id:
candidate.id, name:
candidate.name,
      votes: candidate.votes
    }
  });
});

// GET results
app.get("/results", (req, res) => {
res.json(candidates);
});

// START SERVER const
PORT = 3000;
app.listen(PORT, () => {
```

```
  console.log("Server running on http://localhost:3000");
});
// PUT /candidates/:id → update candidate name
app.put("/candidates/:id", (req, res) => {   const
id = parseInt(req.params.id);
  const { name } = req.body;

  const candidate = candidates.find(c => c.id === id);

  if (!candidate) {
    return res.status(404).json({ error: "Candidate not found" });
  }

  if (!name) {
    return res.status(400).json({ error: "Name is required" });
  }

  candidate.name = name;

  res.json({
    message: "Candidate updated successfully",
    candidate
  });
});
// DELETE /candidates/:id → delete candidate
app.delete("/candidates/:id", (req, res) => {
const id = parseInt(req.params.id);

  const index = candidates.findIndex(c => c.id === id);

  if (index === -1) {        return res.status(404).json({ error:
"Candidate not found" });
  }

  const deletedCandidate = candidates[index];

  //                remove                candidate
candidates.splice(index, 1);

  res.json({
    message:        "Candidate        deleted        successfully",
deletedCandidate
  });
});
```

OUTPUT:

POST http://localhost:3000/vote

Docs Params Authorization Headers 8 Body

raw JSON

```
{
  "userId": "user5",
  "candidateId": 2
}
```

Body

{} JSON Preview Visualize

```
{
    "message": "Vote cast successfully",
    "candidate": {
        "id": 2,
        "name": "Bob",
        "votes": 4
    }
}
```

GET http://localhost:3000/results

Docs Params Authorization Headers 6 Body

Body

{} JSON Preview Visualize

```
[
    {
        "id": 1,
        "name": "Alice",
        "votes": 0
    },
    {
        "id": 2,
        "name": "Bob",
        "votes": 4
    },
    {
        "id": 3,
        "name": "Charlie",
        "votes": 1
    }
]
```

	id	name	votes
0	1	Alice	0
1	2	Bob	4
2	3	Charlie	1

PUT http://localhost:3000/candidates/2

Docs Params Authorization Headers 8 Body Scripts

raw JSON

```
1 {
2   "name": "Bobby"
3 }
```

Body

{} JSON Preview Visualize

```
1 {
2     "message": "Candidate updated successfully",
3     "candidate": {
4         "id": 2,
5         "name": "Bobby",
6         "votes": 0
7     }
8 }
```

DELETE http://localhost:3000/candidates/3

Docs Params Authorization Headers 6 Body Scripts

Query Params

Key	Value
Key	Value

Body

{} JSON Preview Visualize

```
1 {
2     "message": "Candidate deleted successfully"
3     "deletedCandidate": {
4         "id": 3,
5         "name": "Charlie",
6         "votes": 0
7     }
8 }
```

Explanation:

This API allows users to view candidates, cast votes, and check results in a simple voting system. It ensures fairness by restricting each user to only one vote using in-memory validation.

Task 5 – Teacher Management API

Task:

Develop a RESTful API to manage teacher records.

Instructions:

• Endpoints required:

o GET /teachers → List all teachers o POST /teachers → Add a new teacher o PUT /teachers/{id} → Update teacher details o DELETE /teachers/{id} → Remove a teacher • Ensure proper error handling and JSON responses.

Expected Output:

• Functional API for managing teacher information.

PROMPT:

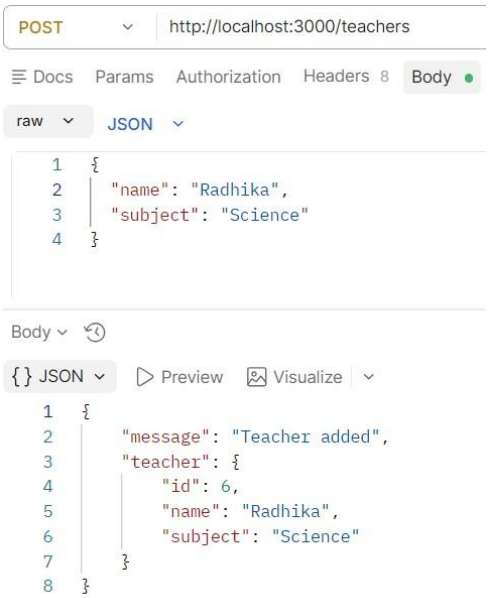
Create a RESTful API for a Teacher Management System using Node.js and Express. The API should include endpoints to list all teachers using GET /teachers, add a new teacher using POST /teachers, update existing teacher details using PUT /teachers/{id}, and delete a teacher using DELETE /teachers/{id}. Ensure proper error handling for cases such as missing data or invalid teacher IDs, and return structured JSON responses for all operations. The system should simulate efficient management of teacher records with full CRUD functionality.

CODE:

```
const express = require("express");
const app = express();

app.use(express.json());

// Teachers let
teachers = [
  { id: 1, name: "Ravi", subject: "Math" },
  { id: 2, name: "Sita", subject: "Science" }
];

// GET
app.get("/teachers", (req, res) => {
res.json(teachers);
});

// POST
app.post("/teachers", (req, res) => {
const { name, subject } = req.body;

  if (!name || !subject) {
    return res.status(400).json({ error: "Name and subject required" });
  }

  const newTeacher = {
id: teachers.length + 1,
name, subject
  };

  teachers.push(newTeacher);

  res.json({ message: "Teacher added", teacher: newTeacher }); });

// PUT app.put("/teachers/:id", (req,
res) => { const id =
parseInt(req.params.id);
  const teacher = teachers.find(t => t.id === id);

  if (!teacher) {
    return res.status(404).json({ error: "Teacher not found" });
}

  const { name, subject } = req.body;
```

```
  if (name) teacher.name = name;
  if (subject) teacher.subject = subject;

  res.json({ message: "Teacher updated", teacher });
});

// DELETE
app.delete("/teachers/:id", (req, res) => {
const id = parseInt(req.params.id);
  const index = teachers.findIndex(t => t.id === id);

  if (index === -1) {
    return res.status(404).json({ error: "Teacher not found" });
  }

  const deleted = teachers.splice(index, 1);

  res.json({ message: "Teacher deleted", deleted });
});

// START SERVER const
PORT = 3000;
app.listen(PORT, () => {
  console.log("Server running on http://localhost:3000");
});
```

OUTPUT:



GET http://localhost:3000/teachers

Docs Params Authorization Headers 6 Bo

Body Cookies Headers 7 Test Results

{ } JSON Preview Visualize


```
[
    {
        "id": 1,
        "name": "Ravi",
        "subject": "Math"
    },
    {
        "id": 2,
        "name": "Sita",
        "subject": "Science"
    },
    {
        "id": 3,
        "name": "Anil",
        "subject": "English"
    },
    {
        "id": 4,
        "name": "Vishal",
        "subject": "Maths"
    },
    {
        "id": 5,
        "name": "Bobby",
        "subject": "Social"
    },
    {
        "id": 6,
        "name": "Radhika",
        "subject": "Science"
    }
]
```


	id	name	subject
0	1	Ravi	Math
1	2	Sita	Science
2	3	Anil	English
3	4	Vishal	Maths
4	5	Bobby	Social
5	6	Radhika	Science

PUT http://localhost:3000/teachers/1

Docs Params Authorization Headers 8 Body

raw JSON

```
{
  "name": "Ravi Kumar"
}
```

Body

{ } JSON Preview Visualize


```
{
    "message": "Teacher updated",
    "teacher": {
        "id": 1,
        "name": "Ravi Kumar",
        "subject": "Math"
    }
}
```

DELETE http://localhost:3000/teachers/2

Docs Params Authorization Headers 6 Body

Query Params

Key	Value
Key	Value

Body

{} JSON Preview Visualize


```
{
    "message": "Teacher deleted",
    "deleted": [
        {
            "id": 2,
            "name": "Sita",
            "subject": "Science"
        }
    ]
}
```


Explanation:

This API allows users to manage teacher records by performing operations like adding, viewing, updating, and deleting teacher details. It ensures proper error handling and returns structured JSON responses, making the system reliable and easy to use.